

과제4 - 집계&ROLLUP

• 과제4

조별 과제 C-3·10점 (집계·원도우)

과제 4 01_seed_core telecom·public

집계 & ROLLUP — 소개·총계

그림 집계와 총계를 만든다

본문 연계 Track 1·Day2 — 집계

① 시나리오 & 사용 테이블

요금제별 활성 가입 현황 리포트를 만들고, 마지막에 전체 총계 행을 붙입니다. 활성 = status = 'ACTIVE' (WHERE로 먼저 거른 뒤 집계). 필터는 개별 행이 아니라 집계 결과에 겁니다(HAVING).

순서 WHERE(행 필터) → GROUP BY(그룹) → HAVING(집계 필터) → ORDER BY

요금제 plan 부모

plan_id	요금제ID	plan_name	요금제명
monthly_fee	월요금	data_gb	데이터

▲ 1: 다 (가입이 요금제를 참조)

가입 subscription 자식

sub_id	가입ID	plan_id	요금제ID(FK)
cust_id	고객ID	status	상태

☞ 가입이 요금제를 참조 — 요금제별 가입 수·평균 월요금 집계 대상.

② 실습 과제·C-3

요금제별 활성 가입자 수와 평균 월요금을 집계하고 HAVING으로 가입자 2명 이상만 남긴 뒤, GROUP BY ROLLUP(plan_id)로 총계 행을 더하고 COALESCE로 총계 행을 '전체'로 라벨링 → 하나의 집계 쿼리로 작성

③ 기대 결과 — 요금제별 가입자수·평균 월요금 + 맨 끝 총계(ROLLUP) 행

🔧 제출 방법

Mac·DBEaver에서 실행 → 작성 SQL + 결과 그리드 캡처 + 해석을 PDF로 저장 및 제출. 형식·평가 기준은 「종합실습 가이드」를 따릅니다.

💡 힌트 — 활성 필터 WHERE status = 'ACTIVE'·집계 조건 HAVING COUNT(*) >= 2·총계행 라벨 COALESCE(plan_id::text, '전체').

1. 요금제별 활성 가입자 수와 평균 월요금 집계
2. HAVING으로 가입자 2명 이상 남기기
3. GROUP BY ROLLUP(plan_id)로 총계 행 더하기
4. COALESCE로 총계 행을 '전체'로 라벨링

• 작성 쿼리

```

-- 1. 요금제 ID가 NULL이면 (총계 행) '전체'로 표시하고, 아니면 문자열로 변환하여 출
COALESCE(s.plan_id::text, '전체') AS plan_id,
-- 2. 해당 요금제를 이용 중인 활성 가입자 수 집계
COUNT(s.sub_id) AS user_count,
-- 3. 해당 요금제의 평균 월요금 계산
AVG(p.monthly_fee) AS avg_monthly_fee
FROM subscription s
-- 4. 가입 정보(subscription)와 요금제 정보(plan)를 plan_id 기준으로 JOIN
JOIN plan p ON s.plan_id = p.plan_id
-- 5. [행 필터링] 해지 고객 제외, 현재 이용 중인('ACTIVE') 가입자만 선별
WHERE s.status = 'ACTIVE'
-- 6. [그룹화 및 총계 생성] 요금제별로 묶어 집계하고, 맨 아래에 전체 합산(ROLLUP) 행 추가
GROUP BY ROLLUP(s.plan_id)
-- 7. [집계 필터링] 가입자가 2명 이상인 그룹만 남김 (단, ROLLUP으로 만든 총계 행은 유지)
HAVING COUNT(s.sub_id) >= 2 OR s.plan_id IS null
-- 8. [정렬] '전체(총계)' 행을 가장 맨 아래로 보내고, 나머지는 plan_id 순으로 정렬
ORDER BY
CASE WHEN s.plan_id IS NULL THEN 1 ELSE 0 END,
s.plan_id;

```

◦ 실행순서

FROM/JOIN(데이터 모으기) → WHERE(행 필터링) → GROUP BY(그룹 묶기) →
HAVING(그룹 필터링) → SELECT(출력 컬럼 계산) → ORDER BY(정렬)

1. FROM/JOIN

```
FROM subscription s
-- 4. 가입 정보(subscription)와 요금제 정보(plan)를 plan_id 기준으로 조인
JOIN plan p ON s.plan_id = p.plan_id
```

- **FROM subscription s** : (구독)테이블을 메인 테이블로 지정, 별칭 s로 지정
- **JOIN plan p** : (요금제)테이블을 가져와서 이전 테이블로 붙이기, 별칭 p로 지정
- **ON s.plan_id = p.plan_id** : 두 테이블에서 plan_id 값이 같은 행끼리 연결

2. WHERE

```
-- 5. [행 필터링] 해지 고객 제외, 현재 이용 중인('ACTIVE') 가입자만 선별
WHERE s.status = 'ACTIVE'
```

- **WHERE s.status = 'ACTIVE'** : 구독 상태가 ACTIVE인 데이터만 골라냄. 해지했거나 일시 정지된 회원은 이후 집계 대상에서 제외

3. GROUP BY

```
-- 6. [그룹화 및 총계 생성] 요금제별로 묶어 집계하고, 맨 아래에 전체 합산(ROLLUP) 행 추가
GROUP BY ROLLUP(s.plan_id)
```

- **GROUP BY ROLLUP(s.plan_id)** : plan_id 별로 데이터를 그룹으로 묶어주는데, ROLLUP 덕분에 각 요금제별 그룹뿐만 아니라 전체 데이터의 합계(총계) 행이 자동으로 생성. 이 때 생성된 총계 행의 s.plan_id 값은 내부적으로 NULL 상태가 된다.

ROLLUP구조 - 컬럼 걷어내는 과정

1단계: 아무것도 안 걷어냄 (GROUP BY s.plan_id)

- 그룹 기준: s.plan_id (요금제별)
- 상태: s.plan_id 라는 묶음 기준이 살아있다.
- 계산 방식:
 - 요금제별로 회원들을 따로 모은다.
 - **COUNT(s.sub_id)** : 해당 요금제 가입자 수
 - **AVG(p.monthly_fee)** : 해당 요금제의 평균 월 요금

2단계: s.plan_id 를 걷어냄 (GROUP BY ())

- 그룹 기준: 없음 (모든 요금제 기준을 뺀)
- 상태: '요금제 구분도 없이 전부 한 덩어리로 합쳐진 상태'가 된다.
- 계산 방식:

- 모든 활성 회원을 하나의 큰 덩어리로 합친다.
- `COUNT(s.sub_id)` : 요금제 구분 없는 **전체 회원 수**
- `AVG(p.monthly_fee)` : 전체 회원이 내는 **전체 평균 월 요금**

	AZ plan_id ▼	123 user_cou ▼	123 avg_mon ▼
1	1	35	19,000
2	2	59	33,000
3	3	74	45,000
4	4	59	55,000
5	5	59	59,000
6	6	66	69,000
7	7	70	89,000
8	8	67	29,000
9	9	60	25,000
10	10	36	39,000
11	전체	585	374.358974359

ROLLUP 후 결과

4. HAVING

— 7. [집계 필터링] 가입자가 2명 이상인 그룹만 남김 (단, ROLLUP으로 만든 총계 행은 유지)
`HAVING COUNT(s.sub_id) >= 2 OR s.plan_id IS null`

- HAVING은 GROUP_BY와 ROLLUP이 모든 계산을 끝낸 뒤 만들어낸 결과표의 각 행을 하나씩 검사하는 필터 역할
- `COUNT(s.sub_id) >= 2` : 해당 요금제 사용하는 가입자 수가 2명 이상인 요금제 그룹만 남김
- `OR s.plan_id IS NULL` : 앞서 ROLLUP 으로 만든 '총계 행'은 `s.plan_id` 가 `NULL` 이므로, 총 가입자 수와 상관없이 조건에서 걸러지지 않고 무조건 결과에 포함되도록 예외 처리를 해준 것.
 OR 조건이 필수 안전장치인 이유 : 만약 전체 시스템의 활성 회원이 1명뿐이거나, 다른 조건에 의해 전체 활성 가입자가 1명만 남은 상황에서는 총계 행을 결과표에서 완전히 삭제해 버림. 전체 총계도 보고 싶어서 ROLLUP을 썼는데, 수가 적다는 이유로 총계 행 자체가 없어지는 것임.
- `COUNT(s.sub_id) >= 2 OR s.plan_id IS NULL` : 가입자가 2명 이상인 행만 남긴다. 하지만 만약 그 행의 plan_id가 NULL인 총계 행이라면 무조건 통과 시켜라

5. SELECT

```

SELECT
  -- 1. 요금제 ID가 NULL이면(총계 행) '전체'로 표시하고, 아니면 문자열로 변환하여 출
  COALESCE(s.plan_id::text, '전체') AS plan_id,
  -- 2. 해당 요금제를 이용 중인 활성 가입자 수 집계
  COUNT(s.sub_id) AS user_count,
  -- 3. 해당 요금제의 평균 월요금 계산
  AVG(p.monthly_fee) AS avg_monthly_fee

```

- `COALESCE(s.plan_id::text, '전체') AS plan_id` :
 - `s.plan_id::text` : 숫자인 ID를 문자열 타입으로 변환
 - `COALESCE(값, '전체')` : ROLLUP 으로 인해 생성된 총계 행의 `plan_id` 가 `NULL` 일 때, 화면에 `NULL` 대신 깔끔하게 '전체'라는 글자로 바뀌어 출력되도록 함
- `COUNT(s.sub_id) AS user_count` : 그룹별(또는 전체) 회원 수를 카운트하여 `user_count` 라는 이름으로 출력
- `AVG(p.monthly_fee) AS avg_monthly_fee` : 그룹별(또는 전체) 회원들의 평균 월 요금을 계산하여 `avg_monthly_fee` 로 출력

6. ORDER BY

```

-- 8. [정렬] '전체(총계)' 행을 가장 맨 아래로 보내고, 나머지는 plan_id 순으로 정렬
ORDER BY
  CASE WHEN s.plan_id IS NULL THEN 1 ELSE 0 END,
  s.plan_id;

```

- `CASE WHEN s.plan_id IS NULL THEN 1 ELSE 0 END` : 일반 요금제 행(`s.plan_id` 가 존재)은 `0` 을 부여하고, 총계 행(`s.plan_id` 가 `NULL`)은 `1` 을 부여
- `s.plan_id` : 상단에 위치한 일반 요금제들끼리는 `plan_id` 순서대로 예쁘게 정렬

결과

	AZ plan_id ▼	123 user_cou ▼	123 avg_mon ▼	
1	1	35	19,000	
2	2	59	33,000	
3	3	74	45,000	
4	4	59	55,000	
5	5	59	59,000	
6	6	66	69,000	
7	7	70	89,000	
8	8	67	29,000	
9	9	60	25,000	
10	10	36	39,000	
11	전체	585	374.358974359	

💡 핵심 포인트 설명

- `WHERE s.status = 'ACTIVE'` : 개별 행을 집계하기 전에 활성 가입자만 거릅니다.
- `GROUP BY ROLLUP(s.plan_id)` : 요금제별 집계와 더불어 맨 아래에 전체를 합산하는 '총계 (NULL)' 행을 자동으로 생성합니다.
- `COALESCE(s.plan_id::text, '전체')` : `ROLLUP` 으로 인해 생성된 NULL 값을 문자열 '전체' 로 바꿔서 보여줍니다.
- `HAVING COUNT(s.sub_id) >= 2` : 그룹화된 결과 중 가입자가 2명 이상인 요금제만 남깁니다. (총계 행이 사라지지 않도록 `OR s.plan_id IS NULL` 조건도 고려해 주시면 안전합니다)

📄 결과 해석

1. **WHERE 절 역할:** `status = 'ACTIVE'` 조건을 통해 해지/비활성 고객을 제외하고 현재 서비스를 이용 중인 가입자만 추출했습니다.
2. **ROLLUP 및 COALESCE 처리:** `GROUP BY ROLLUP(plan_id)` 를 사용해 요금제별 집계와 전체 총계를 한 번에 구했으며, `COALESCE` 함수를 적용해 NULL로 나타나는 총계 행의 ID를 '전체' 라는 직관적인 라벨로 표시했습니다.
3. **HAVING 절 조건:** 활성 가입자 수가 2명 이상인 요금제 데이터만 선별하여 출력했습니다.